

# Disaster recovery planning guide



Last reviewed 2024-07-05 UTC

This document is the first part of a series that discusses [disaster recovery \(DR\)](#) (/solutions/backup-dr) in Google Cloud. This part provides an overview of the DR planning process: what you need to know in order to design and implement a DR plan. Subsequent parts discuss specific DR use cases with example implementations on Google Cloud.

The series consists of the following parts:

- Disaster recovery planning guide (this document)
- [Disaster recovery building blocks](#) (/architecture/dr-scenarios-building-blocks)
- [Disaster recovery scenarios for data](#) (/architecture/dr-scenarios-for-data)
- [Disaster recovery scenarios for applications](#) (/architecture/dr-scenarios-for-applications)
- [Architecting disaster recovery for locality-restricted workloads](#) (/architecture/architecting-disaster-recovery-for-locality-restricted-workloads)
- [Disaster recovery use cases: locality-restricted data analytic applications](#) (/architecture/dr-scenarios-locality-restricted-data-analytics)
- [Architecting disaster recovery for cloud infrastructure outages](#) (/solutions/disaster-recovery/architecture)

Service-interrupting events can happen at any time. Your network could have an outage, your latest application push might introduce a critical bug, or you might have to contend with a natural disaster. When things go awry, it's important to have a robust, targeted, and well-tested DR plan.

With a well-designed, well-tested DR plan in place, you can make sure that if catastrophe hits, the impact on your business's bottom line will be minimal. No matter what your DR needs look like, Google Cloud has a robust, flexible, and cost-effective selection of products and features that you can use to build or augment the solution that is right for you.

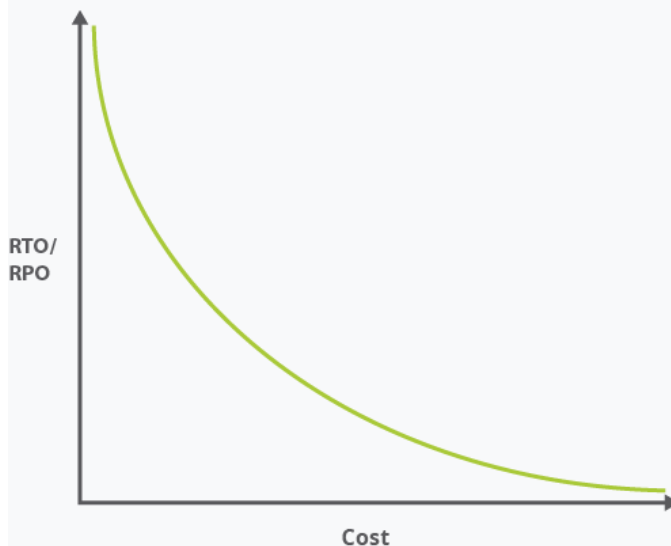
## Basics of DR planning

DR is a subset of business continuity planning

([http://en.wikipedia.org/wiki/Business\\_continuity\\_planning](http://en.wikipedia.org/wiki/Business_continuity_planning)). DR planning begins with a business impact analysis that defines two key metrics:

- A recovery time objective (RTO) ([http://en.wikipedia.org/wiki/Recovery\\_time\\_objective](http://en.wikipedia.org/wiki/Recovery_time_objective)), which is the maximum acceptable length of time that your application can be offline. This value is usually defined as part of a larger service level agreement (SLA) ([http://en.wikipedia.org/wiki/Service-level\\_agreement](http://en.wikipedia.org/wiki/Service-level_agreement)).
- A recovery point objective (RPO) ([http://en.wikipedia.org/wiki/Recovery\\_point\\_objective](http://en.wikipedia.org/wiki/Recovery_point_objective)), which is the maximum acceptable length of time during which data might be lost from your application due to a major incident. This metric varies based on the ways that the data is used. For example, user data that's frequently modified could have an RPO of just a few minutes. In contrast, less critical, infrequently modified data could have an RPO of several hours. (This metric describes only the length of time; it doesn't address the amount or quality of the data that's lost.)

Typically, the smaller your RTO and RPO values are (that is, the faster your application must recover from an interruption), the more your application will cost to run. The following graph shows the ratio of cost to RTO/RPO.



Because smaller RTO and RPO values often mean greater complexity, the associated administrative overhead follows a similar curve. A high-availability application might require you to manage distribution between two physically separated data centers, manage replication, and more.

RTO and RPO values typically roll up into another metric: the service level objective (SLO).

([http://en.wikipedia.org/wiki/Service\\_level\\_objective](http://en.wikipedia.org/wiki/Service_level_objective)), which is a key measurable element of an SLA. SLAs and SLOs are often conflated. An SLA is the entire agreement that specifies what service is to be provided, how it is supported, times, locations, costs, performance, penalties, and responsibilities of the parties involved. SLOs are specific, measurable characteristics of the SLA, such as availability, throughput, frequency, response time, or quality. An SLA can contain many SLOs. RTOs and RPOs are measurable and should be considered SLOs.

You can read more about SLOs and SLAs

(<https://landing.google.com/sre/book/chapters/service-level-objectives.html>) in the Google Site Reliability Engineering book.

You might also be planning an architecture for high availability (HA).

([http://en.wikipedia.org/wiki/High\\_availability](http://en.wikipedia.org/wiki/High_availability)). HA doesn't entirely overlap with DR, but it's often necessary to take HA into account when you're thinking about RTO and RPO values. HA helps to ensure an agreed level of operational performance, usually uptime

(<http://en.wikipedia.org/wiki/Uptime>), for a higher than normal period. When you run production workloads on Google Cloud, you might use a globally distributed system so that if something goes wrong in one region (/docs/geography-and-regions), the application continues to provide service even if it's less widely available. In essence, that application invokes its DR plan.

## Why Google Cloud?

Google Cloud can greatly reduce the costs that are associated with both RTO and RPO when compared to fulfilling RTO and RPO requirements on premises. For example, DR planning requires you to account for a number of requirements, including the following:

- **Capacity:** securing enough resources to scale as needed.
- **Security:** providing physical security to protect assets.
- **Network infrastructure:** including software components such as firewalls and load balancers.
- **Support:** making available skilled technicians to perform maintenance and to address issues.
- **Bandwidth:** planning suitable bandwidth for peak load.
- **Facilities:** ensuring physical infrastructure, including equipment and power.

By providing a highly managed solution on a world-class production platform, Google Cloud helps you bypass most or all of these complicating factors, removing many business costs in the process. In addition, Google Cloud's focus on administrative simplicity means that the costs of managing a complex application are reduced as well.

Google Cloud offers several features that are relevant to DR planning, including the following:

- **A global network.** Google has one of the largest and most advanced computer networks in the world. The Google backbone network uses advanced software-defined networking and edge-caching services to deliver fast, consistent, and scalable performance.
- **Redundancy.** Multiple points of presence (PoPs) across the globe mean strong redundancy. Your data is mirrored automatically across storage devices in multiple locations.
- **Scalability.** Google Cloud is designed to scale like other Google products (for example, search and Gmail), even when you experience a huge traffic spike. Managed services such as Cloud Run, Compute Engine, and Firestore give you automatic scaling that enables your application to grow and shrink as needed.
- **Security.** The [Google security model](/security) (/security) is built on decades of experience with helping to keep customers safe on Google applications like Gmail and Google Workspace. In addition, the site reliability engineering teams at Google help ensure high availability and help prevent abuse of platform resources.
- **Compliance.** Google undergoes regular independent third-party audits to verify that Google Cloud is in alignment with security, privacy, and compliance regulations and best practices. Google Cloud [complies](/security/compliance) (/security/compliance) with certifications such as ISO 27001, SOC 2/3, and PCI DSS 3.0.

## DR patterns

DR patterns are considered to be cold, warm, or hot. These patterns indicate how readily the system can recover when something goes wrong. An analogy might be what you would do if you were driving and punctured a car tire.

How you deal with a flat tire depends on how prepared you are:

- **Cold:** You have no spare tire, so you must call someone to come to you with a new tire and replace it. Your trip stops until help arrives to make the repair.

- Warm: You have a spare tire and a replacement kit, so you can get back on the road using what you have in your car. However, you must stop your journey to repair the problem.
- Hot: You have run-flat tires. You might need to slow down a little, but there is no immediate impact on your journey. Your tires run well enough that you can continue (although you must eventually address the issue).

## Creating a detailed DR plan

This section provides recommendations for how to create your DR plan.

### Design according to your recovery goals

When you design your DR plan, you need to combine your application and data recovery techniques and look at the bigger picture. The typical way to do this is to look at your RTO and RPO values and which DR pattern you can adopt to meet those values. For example, in the case of historical compliance-oriented data, you probably don't need speedy access to the data, so a large RTO value and cold DR pattern is appropriate. However, if your online service experiences an interruption, you'll want to be able to recover both the data and the user-facing part of the application as quickly as possible. In that case, a hot pattern would be more appropriate. Your email notification system, which typically isn't business critical, is probably a candidate for a warm pattern.

For guidance on using Google Cloud to address common DR scenarios, review the application recovery scenarios. These scenarios provide targeted DR strategies for a variety of use cases and offer example implementations on Google Cloud for each.

### Design for end-to-end recovery

It isn't enough just to have a plan for backing up or archiving your data. Make sure your DR plan addresses the full recovery process, from backup to restore to cleanup. We discuss this in the related documents about DR data and recovery.

### Make your tasks specific

When it's time to run your DR plan, you don't want to be stuck guessing what each step means. Make each task in your DR plan consist of one or more concrete, unambiguous commands or

actions. For example, "Run the restore script" is too general. In contrast, "Open a shell and run `/home/example/restore.sh`" is precise and concrete.

## Implementing control measures

Add controls to prevent disasters from occurring and to detect issues before they occur. For example, add a monitor that sends an alert when a data-destructive flow, such as a deletion pipeline, exhibits unexpected spikes or other unusual activity. This monitor could also terminate the pipeline processes if a certain deletion threshold is reached, preventing a catastrophic situation.

## Preparing your software

Part of your DR planning is to make sure that the software you rely on is ready for a recovery event.

### Verify that you can install your software

Make sure that your application software can be installed from source or from a preconfigured image. Make sure that you are appropriately licensed for any software that you will be deploying on Google Cloud—check with the supplier of the software for guidance.

Make sure that needed Compute Engine resources are available in the recovery environment. This might require preallocating instances or [reserving](#) (/compute/docs/instances/choose-reservation-type) them.

### Design continuous deployment for recovery

Your continuous deployment (CD) toolset is an integral component when you are deploying your applications. As part of your recovery plan, you must consider where in your recovered environment you will deploy artifacts. Plan where you want to host your CD environment and artifacts—they need to be available and operational in the event of a disaster.

## Implementing security and compliance controls

When you design a DR plan, security is important. The same controls that you have in your production environment must apply to your recovered environment. Compliance regulations will also apply to your recovered environment.

## Configure security the same for the DR and production environments

Make sure that your network controls provide the same separation and blocking that the source production environment uses. Learn how to configure [Shared VPC](#) (/vpc/docs/provisioning-shared-vpc) and [firewalls](#) (/vpc/docs/firewalls) to let you establish centralized networking and security control of your deployment, to configure subnets, and to control inbound and outbound traffic. Understand how to use service accounts to implement least privilege for applications that access Google Cloud APIs. Make sure to use service accounts as part of the firewall rules.

Make sure that you grant users the same access to the DR environment that they have in the source production environment. The following list outlines ways to synchronize permissions between environments:

- If your production environment is Google Cloud, replicating IAM policies in the DR environment is straightforward. You can use [infrastructure as code \(IaC\)](#) ([http://en.wikipedia.org/wiki/Infrastructure\\_as\\_Code](http://en.wikipedia.org/wiki/Infrastructure_as_Code)) tools like [Terraform](#) (/docs/terraform) to deploy your [IAM](#) (/iam) policies to production. You then use the same tools to bind the policies to corresponding resources in the DR environment as part of the process of standing up your DR environment.
- If your production environment is on-premises, you map the functional roles, such as your network administrator and auditor roles, to IAM policies that have the appropriate IAM roles. The IAM documentation has some example functional role configurations—for example, see the documentation for creating [networking](#) (/iam/docs/job-functions/networking) and [audit logging](#) (/iam/docs/roles-audit-logging) functional roles.
- You have to configure IAM policies to grant appropriate permissions to products. For example, you might want to [restrict access to specific Cloud Storage buckets](#) (/storage/docs/access-control/iam#project-level\_roles\_vs\_bucket-level\_roles).
- If your production environment is another cloud provider, map the permissions in the other provider's IAM policies to Google Cloud IAM policies.

## Verify your DR security

After you've configured permissions for the DR environment, make sure that you test everything. Create a test environment. Verify that the permissions that you grant to users match those that the users have on-premises.

## Make sure users can access the DR environment

Don't wait for a disaster to occur before checking that your users can access the DR environment. Make sure that you have granted appropriate access rights to users, developers, operators, data scientists, security administrators, network administrators, and any other roles in your organization. If you are using an alternative identity system, make sure that accounts have been synced with your Cloud Identity account. Because the DR environment will be your production environment for a while, get your users who will need access to the DR environment to sign in, and resolve any authentication issues. Incorporate users who are logging in to the DR environment as part of the regular DR tests that you implement.

To centrally manage who has administrative access to virtual machines (VMs) that are launched, enable the [OS login](#) (/compute/docs/instances/managing-instance-access) feature on the Google Cloud projects that constitute your DR environment.

## Train users

Users need to understand how to undertake the actions in Google Cloud that they're used to accomplishing in the production environment, such as logging in and accessing VMs. Using the test environment, train your users how to perform these tasks in ways that safeguard your system's security.

## Make sure that the DR environment meets compliance requirements

Verify that access to your DR environment is restricted to only those who need access. Make sure that PII data is redacted and encrypted. If you perform regular penetration tests on your production environment, you should include your DR environment as part of that scope and carry out regular tests by standing up a DR environment.

Make sure that while your DR environment is in service, any logs that you collect are backfilled into the log archive of your production environment. Similarly, make sure that as part of your DR environment, you can export audit logs that are collected through [Cloud Logging](#) (/logging)

to your main log sink archive. Use the export sink facilities. For application logs, create a mirror of your on-premises logging and monitoring environment. If your production environment is another cloud provider, map that provider's logging and monitoring to the equivalent Google Cloud services. Have a process in place to format input into your production environment.

## Treat recovered data like production data

Make sure that the security controls that you apply to your production data also apply to your recovered data: the same permissions, encryption, and audit requirements should all apply.

Know where your backups are located and who is authorized to restore data. Make sure your recovery process is auditable—after a disaster recovery, make sure you can show who had access to the backup data and who performed the recovery.

## Making sure your DR plan works

Make sure that if a disaster does occur, your DR plan works as intended.

## Maintain more than one data recovery path

In the event of a disaster, your connection method to Google Cloud might become unavailable. Implement an alternative means of access to Google Cloud to help ensure that you can transfer data to Google Cloud. Regularly test that the backup path is operational.

## Test your plan regularly

After you create a DR plan, test it regularly, noting any issues that come up and adjusting your plan accordingly. Using Google Cloud, you can test recovery scenarios at minimal cost. We recommend that you implement the following to help with your testing:

- **Automate infrastructure provisioning.** You can use IaC tools like Terraform to automate the provisioning of your Google Cloud infrastructure. If you're running your production environment on premises, make sure that you have a monitoring process that can start the DR process when it detects a failure and can trigger the appropriate recovery actions.
- **Monitor your environments with Google Cloud Observability.** Google Cloud has excellent logging and monitoring tools that you can access through API calls, allowing you to

automate the deployment of recovery scenarios by reacting to metrics. When you're designing tests, make sure that you have appropriate monitoring and alerting in place that can trigger appropriate recovery actions.

- **Perform the testing noted earlier:**
  - Test that permissions and user access work in the DR environment like they do in the production environment.
  - Perform penetration testing on your DR environment.
  - Perform a test in which your usual access path to Google Cloud doesn't work.

## What's next?

- Read about [Google Cloud geography and regions](/docs/geography-and-regions) (/docs/geography-and-regions).
- Read other documents in this DR series:
  - [Disaster recovery building blocks](/architecture/dr-scenarios-building-blocks) (/architecture/dr-scenarios-building-blocks)
  - [Disaster recovery scenarios for data](/architecture/dr-scenarios-for-data) (/architecture/dr-scenarios-for-data)
  - [Disaster recovery scenarios for applications](/architecture/dr-scenarios-for-applications) (/architecture/dr-scenarios-for-applications)
  - [Architecting disaster recovery for locality-restricted workloads](/architecture/architecting-disaster-recovery-for-locality-restricted-workloads) (/architecture/architecting-disaster-recovery-for-locality-restricted-workloads)
  - [Disaster recovery use cases: locality-restricted data analytic applications](/architecture/dr-scenarios-locality-restricted-data-analytics) (/architecture/dr-scenarios-locality-restricted-data-analytics)
  - [Architecting disaster recovery for cloud infrastructure outages](/solutions/disaster-recovery/architecture) (/solutions/disaster-recovery/architecture)
- For more reference architectures, diagrams, and best practices, explore the [Cloud Architecture Center](/architecture) (/architecture).

## Contributors

Authors:

- [Grace Mollison](https://www.linkedin.com/in/grace-mollison-6296614) (https://www.linkedin.com/in/grace-mollison-6296614) | Solutions Lead
- [Marco Ferrari](https://www.linkedin.com/in/ferrarimark) (https://www.linkedin.com/in/ferrarimark) | Cloud Solutions Architect

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (https://www.apache.org/licenses/LICENSE-2.0). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2024-07-05 UTC.